Etapa 7 Revisar la propuesta con SOLID

1. Principio de Responsabilidad Única (SRP — *Single Responsibility Principle*)

- **Principio Evaluado:** S — Cada clase o componente debe tener una única razón para cambiar.

- **Evidencia Encontrada:** En el diseño inicial de la capa de aplicación, el servicio VentaService asumió la responsabilidad de calcular los descuentos automáticos (5% regular y 10% empresarial), evaluar el límite de crédito del cliente y ejecutar la sentencia de persistencia en la base de datos.

- **Impacto / Riesgo:** Violación de SRP. El servicio acumulaba múltiples motivos de cambio independientemente de la orquestación: modificaciones en las reglas comerciales de descuento, cambios en las políticas financieras de cobro/crédito o actualizaciones en la tecnología de base de datos.

- **Cambio Realizado / Decisión Confirmada**: Se refactorizó la responsabilidad del cálculo de descuento delegando directamente al Agregado Cotizacion / Venta, la evaluación del crédito al Agregado ClienteEmpresarial y la gestión de la base de datos a las interfaces IVentaRepository y IUnitOfWork. El VentaService quedó acotado exclusivamente a coordinar el flujo del caso de uso sin absorber reglas de negocio

2. Principio de Abierto / Cerrado (OCP — *Open / Closed Principle*)

- **Principio Evaluado:** O — Las entidades de software deben estar abiertas a la extensión, pero cerradas a la modificación.

- **Evidencia Encontrada:** La lógica para calcular la fecha de vencimiento de cotizaciones (5 días para cliente regular y 3 días para cliente empresarial o mayoreo) dependía de una estructura condicional if-else o switch que evaluaba cadenas de texto fijas con el tipo de cliente.

- **Impacto / Riesgo:** Violación de OCP. Si FerreMax introduce un nuevo perfil comercial en el futuro (ej. "Distribuidor VIP" o "Gobierno") con reglas de vigencia o descuentos personalizados, se tendría que modificar el código interno de la clase Cotizacion, incrementando el riesgo de introducir errores en código probado.

- **Cambio Realizado / Decisión Confirmada**: Se abstrajo la regla de vigencias y descuentos mediante Value Objects polimórficos de políticas de cliente (PoliticaVigencia, PoliticaDescuento). De este modo, la incorporación de nuevos perfiles de cliente se realiza extendiendo e implementando nuevas políticas sin editar la clase Cotizacion existente.

3. Principio de Sustitución de Liskov (LSP — Liskov Substitution Principle)

- **Principio Evaluado:** L — Las clases derivadas deben poder sustituir a sus clases base sin alterar el comportamiento del sistema

- **Evidencia Encontrada:** Al diseñar la jerarquía entre la clase base Cliente y la clase derivada ClienteEmpresarial, la clase base incluía el método validarLineaCredito(). Sin embargo, al ejecutar este método sobre un ClienteParticular (venta de mostrador de contado), la clase arrojaba una excepción no controlada (NotSupportedException) porque los clientes particulares no manejan crédito ni saldo vencido

- **Impacto / Riesgo:** Violación de LSP. Sustituir un Cliente genérico por un ClienteParticular alteraba el contrato esperado e interrumpía el flujo de cobro inmediato en caja.

- **Cambio Realizado / Decisión Confirmada**: Se ajustó la jerarquía eliminando los métodos de crédito de la clase base Cliente. La clase Cliente únicamente conserva atributos globales de identificación, mientras que las reglas de línea de crédito, facturas vencidas y el estado de "Crédito suspendido" quedaron confinadas de forma segura dentro del Agregado ClienteEmpresarial.

4. Principio de Segregación de Interfaces (ISP — Interface Segregation Principle)

- **Principio Evaluado:** I — Ningún componente debe verse obligado a depender de métodos que no utiliza.

- **Evidencia Encontrada:** Se propuso inicialmente una interfaz de repositorio monolítica llamada IFerreMaxRepository que agrupaba todas las operaciones de consulta y almacenamiento del sistema (guardarVenta(), consultarStockSKU(), autorizarExcepcionCredito(), generarRutaLogistica()).

- **Impacto / Riesgo:** Violación de ISP. Componentes especializados como RuteoLogisticoService o CreditoService terminaban dependiendo de una interfaz gigante con decenas de métodos irrelevantes para su función operativa.

- **Cambio Realizado / Decisión Confirmada**: Se dividió la interfaz masiva en repositorios específicos enfocados en cada Agregado y contexto de negocio: IVentaRepository, IInventarioRepository, IClienteRepository e IHojaDeRutaRepository. Cada servicio de aplicación únicamente consume la interfaz acotada que requiere.

5. Principio de Inversión de Dependencias (DIP — Dependency Inversion Principle)

- **Principio Evaluado:** D — Los módulos de alto nivel no deben depender de módulos de bajo nivel; ambos deben depender de abstracciones.

- **Evidencia Encontrada:** En las primeras iteraciones, los servicios de la capa de Aplicación (CotizacionService, ReservaStockService) instanciaban directamente clases de infraestructura concretas (ej. MySQLVentaRepository o SqlServerDbContext).

- **Impacto / Riesgo:** Violación de DIP. La capa de Aplicación y las reglas del Dominio quedaban fuertemente acopladas al motor de base de datos relacional, impidiendo realizar pruebas unitarias independientes y dificultando la implementación del modo offline en sucursal.

- **Cambio Realizado / Decisión Confirmada**: Se aplicó inversión de dependencias mediante la inyección de abstracciones. El Dominio y la Aplicación dependen exclusivamente de la interfaz IVentaRepository y de la abstracción IUnitOfWork, mientras que las implementaciones tecnológicas de base de datos residen en la capa de Infrastructure y se inyectan en tiempo de ejecución